CloudCost.

Do prompt à entrega controlada.

Um estimador de custos de nuvem, uma entrega na Azure e uma cadeia de evidências.

> IA como ferramenta de criação.
> **Não como dependência da aplicação.**



Sem chave de IA, modelo remoto ou consumo de tokens em runtime.

Quanto custa este cenário por mês?

Compare computação, armazenamento e tráfego usando preços informados por você.

Exemplo didático em USD

Componente	Cálculo	Subtotal
Computação	1 × 720 h × US$ 0,10/h	US$ 72,00
Armazenamento	100 GB × US$ 0,10/GB	US$ 10,00
Saída de dados	50 GB × US$ 0,10/GB	US$ 5,00

Total = computação + armazenamento + saída de dados

Armazenamento e tráfego consideram as taxas do mês simulado. Valores apresentados com duas casas decimais.

ESTIMATIVA MENSAL

US$ 87,00

Preços ilustrativos, não cotações de AWS, Azure ou Google Cloud.

Limite do modelo: não inclui impostos, descontos, câmbio, região ou todos os serviços. Não é uma fatura nem garantia de precisão de cobrança.

Destino Azure, desenvolvimento local.

Container Apps Consumption + ACR privado; a mesma aplicação stateless também roda com Compose.



Build multi-stage

Go 1.27.3 no builder e na CI. HTML/CSS/JS incorporados ao binário, sem bibliotecas externas de aplicação.

Compose para o demo local

Navegador → Go → scratch. Publicação em `127.0.0.1:8080:8080`, sem banco, cookies ou telemetria.

Azure como destino

`infra/azure/main.bicep` declara recursos. HTTPS no domínio padrão. Configuração não comprova provisionamento.

Fontes: Azure Container Apps • managed identity / ACR • Docker

Reduzir superfície, limitar impacto.

Controles definidos em código e configuração — não uma declaração de invulnerabilidade.

01

Entrada e API

- JSON estrito: campos obrigatórios, tipos e faixas definidos.
- Rejeita nulos, extras, valores negativos e JSON excedente.
- Corpo limitado a **4.096 bytes**.
- Erros `400 / 413 / 415 / 405` e resposta JSON previsível.

02

Navegador e dados

- CSP com scripts e estilos locais; sem inline ou CDN.
- `frame-ancestors 'none'` e `nosniff`.
- `Referrer-Policy: no-referrer` e permissões restritas.
- Sem persistência, cookies, telemetria ou logs de entradas.

03

Runtime local / Compose

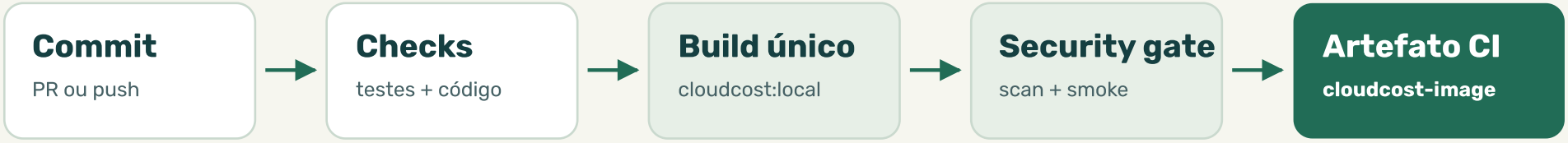
- `scratch`, usuário **65532** e filesystem somente leitura.
- `cap_drop: ALL` e `no-new-privileges`.
- Limites de recursos e healthcheck pelo próprio binário.
- Porta publicada somente no loopback.

> **Fronteira importante:** os controles do Compose não são automaticamente equivalentes na Azure. Scratch não elimina falhas de Go, código, kernel ou plataforma; revisar cada ambiente.

Fontes: OWASP ASVS • Docker Compose / services

Construir uma vez. Verificar antes de publicar.

Fluxo configurado em `.github/workflows/ci.yml`; execução deve ser comprovada por run e relatórios.



Código e segredos

go vet + testes, gosec (SAST), govulncheck (SCA/stdlib) e Gitleaks.

Imagem e configuração

Trivy: **HIGH/CRITICAL** na imagem, inclusive sem correção; misconfigurações em todas as severidades.

Artefato e comportamento

SBOM CycloneDX + smoke/DAST HTTP. A imagem examinada é a mesma executada e promovida.

> **Falha em gate bloqueia promoção.** CI aprovada → `deploy-azure.yml` manual com OIDC → ACR privado → Container Apps por `@sha256:digest`. Sem rebuild ou senha Azure armazenada.

Fontes: GitHub Actions / secure use • Azure / OpenID Connect • configuração ≠ execução

Um scanner não é um atestado.

O resultado depende do alvo, da ferramenta, da base de vulnerabilidades e da data.

1 **Identificar**

Fixar commit, imagem/digest, versões e data da execução. Gerar inventário/SBOM.

2 **Analisar**

Cruzar SAST, SCA, segredos, configuração e testes HTTP. govulncheck considera chamadas alcançáveis.

3 **Tratar e reavaliar**

Examinar impacto e exposição; corrigir a origem. Reexecutar os gates e preservar os resultados.

COMO LER O ESTADO

Configurado ≠ executado

Configurado	Scripts, workflow e critérios versionados.
Executado	Comando, saída, retorno, data e commit/digest em relatório.
Não atestado	Sem afirmação de scans aprovados, CI executada ou ausência de falhas.

Evidências: `reports/`, docs/evidencias.md e run em Actions. Azure implantada só com registro próprio.

> **Zero achados não significa zero falhas.** Pode haver cobertura incompleta, falsos negativos ou CVEs ainda não publicados. Smoke HTTP não substitui pentest.

Fontes: Go / vulnerability management • Trivy / vulnerability scanning

Mapeamento educativo. Não certificação.

Práticas selecionadas ajudam a organizar controles; a avaliação integral depende do contexto real.

Referências e aplicação no projeto

Referência	Contribuição	Evidência a examinar
NIST SSDF	Proteger software, produzir código seguro e responder a vulnerabilidades.	Revisão, gates, inventário e processo de correção.
OWASP ASVS	Validação, headers e configuração segura da aplicação.	Implementação + testes dos controles selecionados.
Minimização de dados	Não persistir entradas, usar cookies ou registrar corpos de requisição.	Fluxo de dados e revisão de logs da aplicação e do ambiente.

Detalhes, critérios e limitações: docs/conformidade.md.

RESPONSABILIDADE OPERACIONAL

Na Azure, o contexto importa.

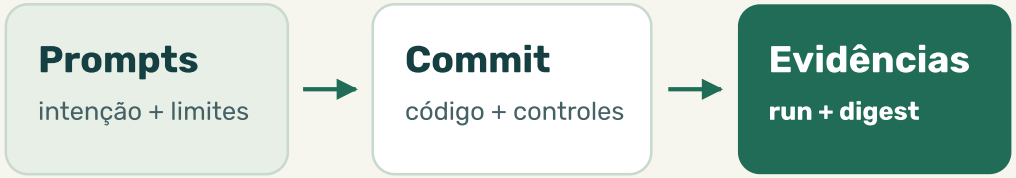
- Aprovar assinatura, região e orçamento.
- Federar OIDC e limitar RBAC.
- Validar HTTPS, revisão ativa e acesso ao ACR.
- Definir logs, retenção e resposta a incidentes.
- ACR e serviços habilitados podem gerar custos mesmo com zero réplicas.

> **Sem selo automático:** não há declaração de conformidade integral com ASVS, SSDF, LGPD, ISO 27001 ou SOC 2. Até endereços IP em logs do proxy exigem avaliação de privacidade.

Fontes: NIST SP 800-218 / SSDF 1.1 • OWASP ASVS 5.0

A entrega inclui o caminho até o código.

Prompts tornam decisões visíveis. Git, relatórios e digest conectam intenção e artefato.



Reproduzir o processo não garante a mesma saída de um modelo de IA.

Arquivo de prompts

Leia o contexto, as restrições e as decisões. Diferencie registro da interação de prompts reutilizáveis; não exponha credenciais.

Abrir arquivo local: prompts/README.md

Abrir prompts no GitHub (exige push e acesso)

REPOSITÓRIO REAL · PRIVADO

Publicação com responsabilidade

1. Revisar e fazer push para `main`; exigir os checks na política de revisão.
2. Habilitar Actions e configurar ambiente de deploy, OIDC e permissões Azure.
3. Selecionar uma CI aprovada no deploy manual, sem reconstruir a imagem.
4. Guardar run, commit e digest; conceder acesso ao público da apresentação.

github.com/felipinhobgtom/cloudcost-devsecops

A URL foi criada. Push, CI e implantação Azure precisam de comprovação própria; nenhum provisionamento é presumido.

Do demo local à entrega Azure.

Local: Docker Engine + Compose v2 e Python 3. Cloud: assinatura, permissões e custos aprovados.

DEMO LOCAL / RAIZ DO REPOSITÓRIO

```
# Construir e aguardar o healthcheck
docker compose up --build -d --wait

# Inspecionar a API e o comportamento HTTP
curl -fsS http://127.0.0.1:8080/healthz
python3 scripts/smoke.py http://127.0.0.1:8080

# Executar controles locais e salvar relatórios
bash scripts/security-checks.sh
```

Aplicação: http://127.0.0.1:8080

Esperado: `{"status":"ok"}` e US$ 87,00 no exemplo. Não são capturas de execução. Build e scanners exigem downloads; pré-requisitos no README.

DESTINO / AZURE CONTAINER APPS

Promover com evidência

1. Executar o bootstrap com assinatura, região, permissões e aceite de custos.
2. `gh workflow run deploy-azure.yml --ref main -f run_id="$CI_RUN_ID"`
3. Conferir digest no ACR, revisão ativa e `/healthz` HTTPS; guardar os relatórios.

> Selecionar uma CI aprovada. Roteiro: docs/azure.md. Configurado não significa provisionado.

Encerrar local: `docker compose down`
Na Azure: revisar custos e remover recursos dedicados quando não forem mais necessários.